

The Killer Document

Kripke-Iris Logical Relations (KILR)

Benjamin Peters

May 28, 2026

This document is intended to be a technical introduction to Kripke-Iris logical relations (KILR) and its underlying separation logic, CROCUS. I'm afraid it is neither a gentle nor an easy to read introduction: it contains many definitions, rules, and figures that need to be pondered for a while.

Interestingly, while writing this, I didn't feel like I needed to include many proofs: They're all either easy or standard. This, of course, means they make for great exercises! You should do the exercises interspersed with the text.

Contents

1	Region types	3
2	Monotone predicates	6
2.1	Objective Invariants	7
2.2	Why Monotone Predicates aren't enough	10
3	Crocus	11
4	A Kripke-Iris logical relation for region types	11
4.1	Looking ahead	13
4.2	Comparing the region types LR and KILR	14
5	A KILR logical relation for OCaml's portability axis using thread-local invariants	14
5.1	Introduction to PORTCAML	14
5.1.1	Portability and Contention	15
5.1.2	Capsules as Isolated Regions	17
5.1.3	PORTCAML on paper	17
5.1.4	Operational Semantics and Program Logic	19
5.2	A standard LR for PORTCAML	19
5.3	Crocus logic	21
5.4	KILR	22
5.5	Model of the Crocus logic	23
5.6	Adding atomic references	23
5.7	Adding locks	23

6	Why worlds are inevitable	23
7	How to model modes?	23
8	MonPred vs ParamPred	23
9	OPEN: A logical relation for PortSharedCaml	23
10	Thread-local invariants and masks	23
11	TODO: A KILR logical relation for OCaml's borrowing using local invariants	23
12	KILR and lenses	24
13	TODO: KILR and public/private transitions	24
14	TODO: A KILR logical relation for OCaml local state	24
15	TODO: OCaml with split axes	24
16	AME, and what does it have to do with anything	24
	References	26

List of Figures

1	Axiomatic semantics for a language with regions.	3
2	REGLANG typing judgement.	4
3	The basic connectives of monotone Iris predicates.	6
4	Reasoning rules for objectively and subjectively.	8
5	Rules for Objective propositions.	9
6	Rules for at-least and at-world.	10
7	Definition and rules for objective invariants.	10
8	The CROCUS extension of MONPRED.	11
9	PORTCAML typing judgement.	18
10	Iris rules for token invariants.	20
11	PortCaml thread-local invariant rules.	22
12	Masked thread-local invariant rules.	24
13	Surface syntax and typing judgement for AME.	27

$$\begin{array}{c}
\text{REGWPVALUE} \quad \frac{\Phi \ v}{\text{wp } v \ \{\Phi\}} \qquad \text{REGWPPURESTEP} \quad \frac{e \xrightarrow{\text{pure}} e' \quad \text{wp } e' \ \{\Phi\}}{\text{wp } e \ \{\Phi\}} \qquad \text{REGWPBIND} \quad \frac{\text{wp } e \ \{v. \text{wp } K[v] \ \{\Phi\}\}}{\text{wp } K[e] \ \{\Phi\}} \\
\\
\text{REGWPREGION} \quad \frac{\forall \rho. \text{is-alive } \rho \multimap \text{wp } e \ \{v. \text{is-alive } \rho \multimap \Phi \ v\}}{\text{wp region } e \ \{\Phi\}} \qquad \text{REGWPALLOC} \quad \frac{\text{is-alive } \rho \multimap (\forall \ell. (\rho, \ell) \mapsto v \multimap \text{is-alive } \rho \multimap \Phi(\rho, \ell))}{\text{wp alloc}_\rho v \ \{\Phi\}} \\
\\
\text{REGWPLoad} \quad \frac{\text{is-alive } \rho \multimap (\rho, \ell) \mapsto v \multimap (\text{is-alive } \rho \multimap (\rho, \ell) \mapsto v \multimap \Phi \ v)}{\text{wp } !(\rho, \ell) \ \{\Phi\}} \\
\\
\text{REGWPSTORE} \quad \frac{\text{is-alive } \rho \multimap (\rho, \ell) \mapsto v \multimap (\text{is-alive } \rho \multimap (\rho, \ell) \mapsto w \multimap \Phi \ ())}{\text{wp } (\rho, \ell) \leftarrow w \ \{\Phi\}}
\end{array}$$

Figure 1: Axiomatic semantics for a language with regions.

1 Region types

Consider the axiomatic semantics of a language with *regions* **REGLANG** in Figure 1. The exclusive resource **is-alive** ρ states that region ρ is still alive. The region e construct allocates a new region for the duration of e and deallocates it afterwards. Crucially, when a region e ends, we need to return the knowledge **is-alive** ρ that the corresponding region is still alive, to ensure that it cannot be accessed anymore. References are associated with a region, and are deallocated once their region is popped. In that case, they cannot be accessed anymore, because their **is-alive** ρ isn't available anymore.

Now, ponder Figure 2. How would you prove soundness of this type system?

Really, do ponder. Don't read ahead. This text will ask you to ponder many times, and you should follow its instructions.

A logical relation for region types, somehow, needs to ensure that for each active region $\rho \in \sigma$ it tracks access to a token **is-alive** ρ . Critically, **is-alive** ρ cannot be (naively) stored in an invariant, or we will never be able to restore it in the postcondition of **REGWPREGION**.

$A, B : \text{Type} ::= \mathbb{1} \mid A \times B \mid A + B \mid A \xrightarrow{\sigma} B \mid \text{ref}_{\rho} A$
 $x, y, \dots \in \text{Var}$
 $\rho_1, \rho_2, \dots \in \text{RegVar}$
 $e : \text{Expr} ::= x \mid () \mid (e_1, e_2) \mid \pi_i e \mid \text{inj}_i e$
 $\quad \mid \text{case } e \text{ of } \text{inj}_1 x \Rightarrow e_1 \mid \text{inj}_2 y \Rightarrow e_2$
 $\quad \mid \text{rec } f(x).e \mid e_1 e_2 \mid \text{alloc}_{\rho} e \mid !e \mid e_1 \leftarrow e_2 \mid \text{region } e$
 $\sigma \in \mathcal{P}(\text{RegVar})$
 $\Gamma ::= \text{Var} \rightarrow_{\text{fin}} \text{Type}$

$\boxed{\Gamma \vdash e : A @ \sigma}$

$\frac{\text{REGTVAR} \quad (x : A) \in \Gamma}{\Gamma \vdash x : A @ \emptyset}$	$\frac{\text{REGTUNIT}}{\Gamma \vdash () : \mathbb{1} @ \emptyset}$	$\frac{\text{REGTPAIR} \quad \Gamma \vdash e_1 : A_1 @ \sigma_1 \quad \Gamma \vdash e_2 : A_2 @ \sigma_2}{\Gamma \vdash (e_1, e_2) : A_1 \times A_2 @ \sigma_1 \cup \sigma_2}$
$\frac{\text{REGTPROJ} \quad \Gamma \vdash e : A_1 \times A_2 @ \sigma}{\Gamma \vdash \pi_i e : A_i @ \sigma}$	$\frac{\text{REGTINJ} \quad \Gamma \vdash e : A_i @ \sigma}{\Gamma \vdash \text{inj}_i e : A_1 + A_2 @ \sigma}$	
$\frac{\text{REGTCASE} \quad \Gamma \vdash e : A_1 + A_2 @ \sigma \quad \Gamma, x : A_1 \vdash e_1 : B @ \sigma' \quad \Gamma, y : A_2 \vdash e_2 : B @ \sigma'}{\Gamma \vdash \text{case } e \text{ of } \text{inj}_1 x \Rightarrow e_1 \mid \text{inj}_2 y \Rightarrow e_2 : B @ \sigma \cup \sigma'}$		
$\frac{\text{REGTREC} \quad \Gamma, f : A \xrightarrow{\sigma} B, x : A \vdash e : B @ \sigma}{\Gamma \vdash \text{rec } f(x).e : A \xrightarrow{\sigma} B @ \emptyset}$	$\frac{\text{REGTAPP} \quad \Gamma \vdash e_1 : A \xrightarrow{\sigma} B @ \sigma_1 \quad \Gamma \vdash e_2 : A @ \sigma_2}{\Gamma \vdash e_1 e_2 : B @ \sigma_1 \cup \sigma_2 \cup \sigma}$	
$\frac{\text{REGTNEW} \quad \Gamma \vdash e : A @ \sigma}{\Gamma \vdash \text{alloc}_{\rho} e : \text{ref}_{\rho} A @ \sigma \cup \{\rho\}}$	$\frac{\text{REGTDEREF} \quad \Gamma \vdash e : \text{ref}_{\rho} A @ \sigma}{\Gamma \vdash !e : A @ \sigma \cup \{\rho\}}$	
$\frac{\text{REGTASSIGN} \quad \Gamma \vdash e_1 : \text{ref}_{\rho} A @ \sigma_1 \quad \Gamma \vdash e_2 : A @ \sigma_2}{\Gamma \vdash e_1 \leftarrow e_2 : \mathbb{1} @ \sigma_1 \cup \sigma_2 \cup \{\rho\}}$	$\frac{\text{REGTMASK} \quad \forall \rho. \Gamma \vdash e : A @ \sigma \cup \{\rho\}}{\Gamma \vdash \text{region } e : A @ \sigma}$	
$\frac{\text{REGTSUB} \quad \Gamma \vdash e : A @ \sigma \quad \sigma \subseteq \sigma'}{\Gamma \vdash e : A @ \sigma'}$		

Figure 2: REGLANG typing judgement.

Instead of letting you suffer for much longer, here is a potential Iris logical relation.

$$\begin{aligned}
\text{SemType} &\triangleq \{P : \text{Val} \rightarrow i\text{Prop} \mid \forall v. \text{Persistent}(Pv)\} \\
\llbracket \mathbf{1} \rrbracket v &\triangleq v = () \\
\llbracket A \times B \rrbracket v &\triangleq \exists v_1, v_2. v = (v_1, v_2) \wedge \llbracket A \rrbracket v_1 * \llbracket B \rrbracket v_2 \\
\llbracket A + B \rrbracket v &\triangleq (\exists u. v = \text{inj}_1 u \wedge \llbracket A \rrbracket u) \vee (\exists u. v = \text{inj}_2 u \wedge \llbracket B \rrbracket u) \\
\llbracket \text{ref}_\rho A \rrbracket v &\triangleq \exists \ell. v = (\rho, \ell) \wedge \boxed{\exists u. (\rho, \ell) \mapsto u * \llbracket A \rrbracket u}^{(\rho, \ell)} \\
\llbracket A \xrightarrow{\sigma} B \rrbracket v &\triangleq \Box \forall u. \llbracket A \rrbracket u * \mathcal{E}[\llbracket B \rrbracket^\sigma](vu) \\
\mathcal{I}[\sigma] &\triangleq \bigstar_{\rho \in \sigma} \text{is-alive } \rho \\
\mathcal{E}[\llbracket A \rrbracket^\sigma] e &\triangleq \forall \sigma' \supseteq \sigma. \mathcal{I}[\sigma'] * \text{wp } e \{v. \mathcal{I}[\sigma'] * \llbracket A \rrbracket v\} \\
\mathcal{G}[\Gamma] \gamma &\triangleq \bigstar_{x \in \text{dom } \Gamma} \llbracket \Gamma(x) \rrbracket (\gamma(x)) \\
\Gamma \models e : A @ \sigma &\triangleq \forall \gamma. \mathcal{G}[\Gamma] \gamma * \mathcal{E}[\llbracket A \rrbracket^\sigma](\gamma(e))
\end{aligned}$$

Note that:

- The expression relation is parametrized by a set of regions σ required by e . This parameter is instantiated by both the function value interpretation and by the semantic typing judgement.
- The expression relation is *upwards closed* with respect to σ .
- The remainder of the logical relation is pretty standard.

Overall, we'd like to obtain the following two standard theorems:

Theorem 1.1 (Soundness). *If $\Gamma \vdash e : A @ \sigma$ then $\Gamma \models e : A @ \sigma$.*

Theorem 1.2 (Adequacy). *If $\emptyset \models e : A @ \emptyset$, then e is safe.*

Ponder these, too.

Exercise 1.3. *Really, do ponder. Specifically:*

- Prove the compatibility lemma for REGTNEW. Where do you get the is-alive ρ precondition for REGWPALLOC from?*
- Prove the compatibility lemma for REGTMASK. How does $\mathcal{I}[\sigma]$ change?*
- Prove the compatibility lemma for REGTSUB. Is the upwards closure $\forall \sigma' \supseteq \sigma. \dots$ in the definition of $\mathcal{E}[\llbracket A \rrbracket^\sigma]$ necessary for the proofs of other rules?*
- Prove the adequacy theorem (Theorem 1.2), assuming adequacy of the weakest precondition: $\text{wp } e \{\Phi\}$ implies that e is safe.*

Overall, this looks like a fine logical relation. Why change it?

$$\begin{aligned}
\text{MONPRED}_{\mathcal{W}} &\triangleq \{\Phi : \mathcal{W} \rightarrow iProp \mid \forall w_1. \forall w_2 \sqsupseteq w_1. \Phi w_1 \vdash \Phi w_2\} \\
\Phi \vdash \Psi &\triangleq \forall w. \Phi w \vdash \Psi w \\
\varphi &\triangleq \lambda _. \varphi \quad (\text{where } \varphi \text{ is a pure Rocq proposition}) \\
\Box \Phi &\triangleq \lambda w. \Box \Phi w \\
\triangleright \Phi &\triangleq \lambda w. \triangleright \Phi w \\
E_1 \Vdash_{E_2} \Phi &\triangleq \lambda w. E_1 \Vdash_{E_2} \Phi w \\
\Phi \wedge \Psi &\triangleq \lambda w. \Phi w \wedge \Psi w \\
\Phi \vee \Psi &\triangleq \lambda w. \Phi w \vee \Psi w \\
\Phi * \Psi &\triangleq \lambda w. \Phi w * \Psi w \\
\Phi \multimap \Psi &\triangleq \lambda w_1. \forall w_2 \sqsupseteq w_1. \Phi w_2 \multimap \Psi w_2 \\
\Phi \Rightarrow \Psi &\triangleq \lambda w_1. \forall w_2 \sqsupseteq w_1. \Phi w_2 \Rightarrow \Psi w_2 \\
[P] &\triangleq \lambda _. P \\
[a]^\gamma &\triangleq \lambda _. [a]^\gamma \\
\uparrow w &\triangleq \lambda w'. w \sqsubseteq w' \\
@_w \Phi &\triangleq \lambda _. \Phi w \\
\langle \text{obj} \rangle \Phi &\triangleq \lambda _. \forall w. \Phi w \\
\langle \text{subj} \rangle \Phi &\triangleq \lambda _. \exists w. \Phi w
\end{aligned}$$

Figure 3: The basic connectives of monotone Iris predicates.

Take a closer look at the definition of $\mathcal{E}[A]^\sigma$. This pattern of “take some resource, execute a WP or an update, and return some resource” is *extremely* common in the Iris literature, both in weakest preconditions and as part of resource updates:

$$\begin{aligned}
R(x) \multimap \text{wp } v \{v. R(y) * \Phi(v)\} \\
R(x) \multimap R(y) * P
\end{aligned} \tag{1}$$

(TODO: Cite sources for this claim) I claim that this pattern is better analyzed in a bespoke logic, CROCUS, with a novel definition of update following the structure of (1). The model of CROCUS is a Kripke semantics in separation logic: it associated a Kripke world with separation logic state. Separately, I claim that CROCUS forms an expressive framework for defining logical relations. We call this framework *Kripke-Iris logical relations*, or KILR. Let’s set up CROCUS, and translate the logical relation we just looked at into a KILR.

2 Monotone predicates

In this section we recall the definition of the logic of *monotone Iris predicates*. Let $(\mathcal{W}, \sqsubseteq)$ be a non-empty pre-order with a least element $\emptyset$ called the *empty world*. The propositions in the logic of *monotone Iris predicates* over $\mathcal{W}$, denoted by $\text{MONPRED}_{\mathcal{W}}$, are *upwards-closed*

predicates. It is defined in Figure 3. In this document, we color **MONPRED** (and later also **CROCUS**) propositions **purple**. **CROCUS** will reuse these connectives unchanged, and add only the Kangaroo update modality and a lifted weakest precondition in Figure 8.

Most logical primitives lift pointwise for this setting, save for some notable exceptions. $\neg*$ and $\implies$ must be *upwards closed*, or they would not lie in $\text{mProp}_{\mathcal{W}}$. The *at-least* proposition $\uparrow w$ and the *at-world* proposition $@_w \Phi$ expose the current world as logical propositions/modalities. The *objectively* $\langle \text{obj} \rangle \Phi$ and *subjectively* $\langle \text{subj} \rangle \Phi$ modalities express that a proposition does not depend on the current world, either because it is truly independent, or because we only know it is valid in some other world.

The notion of Persistent, Timeless, etc. propositions lifts to **MONPRED** cleanly. We also get a new, interesting class of propositions: the *objective* propositions. A proposition Φ is *objective* if it does not depend on the current world, i.e. if $\Phi \vdash \langle \text{obj} \rangle \Phi$ (or equivalently if $\langle \text{subj} \rangle \Phi \vdash \Phi$).

Now, ponder Figures 4 to 6 for the rules of **MONPRED**. Personally, my mental model of **MONPRED** is very close to its actual semantics, and I use mostly use the semantics for my mental reasoning. However, I still feel that sometimes it's helpful to look at these rules to recall some of the more subtle interactions.

Ben: There are not many rules about at-least and at-world here. Are the ones I show enough?

2.1 Objective Invariants

(This section does not need to be read yet if you're reading this document linearly. However, it makes for a good exercise in what it's like to work with **MONPRED**.)

One major disadvantage of **MONPRED** is that the notion of invariant doesn't lift cleanly from Iris. How would we even define a notion of invariant? Initially, one might guess:

$$\text{oinv}_N(\Phi) \stackrel{?}{=} \lambda w. [\text{inv}_N(\Phi w)]$$

This looks good, but it is not upwards closed! With this definition, $\text{oinv}_N(\Phi)$ does not actually lie in **MONPRED**. The reader might propose another fix: We can add an explicit *closure* around the definition to force it to be upwards closed. Consider these two alternative definitions:

$$\begin{aligned} \text{oinv}_N(\Phi) &\stackrel{?}{=} \lambda w. \forall w' \supseteq w. [\text{inv}_N(\Phi w')] \\ \text{oinv}_N(\Phi) &\stackrel{?}{=} \lambda w. \exists w' \subseteq w. [\text{inv}_N(\Phi w')] \end{aligned}$$

Both of these potential definitions are actually upwards closed (exercise: prove this!). Sadly, neither are what we want. The first cannot be allocated: we'd have to allocate an invariant at all future worlds. The latter is a bit more subtle (and we'll actually see it again in a later section!), but still not quite what we want: Let's say we want to prove a standard accessor lemma like in Iris.

$$\text{oinv}_N(\Phi) \xRightarrow{E \Rightarrow E \uparrow N} \Phi * (\Phi \xRightarrow{E \uparrow N \Rightarrow E} \text{Emp}) \quad (2)$$

This doesn't hold, even for the second definition. To see why, let's unfold the $\text{oinv}_N(\Phi)$ and **MONPRED** definitions. Evaluated at world w_1 , (2) becomes:

$$\begin{aligned} \forall w_2 \supseteq w_1. (\exists w'_1 \subseteq w_2. [\text{inv}_N(\Phi w'_1)]) \\ \xRightarrow{E \Rightarrow E \uparrow N} \exists w_3 \supseteq w_2. \Phi w_3 * \Phi \xRightarrow{E \uparrow N \Rightarrow E} \text{Emp } w_3 \end{aligned} \quad (3)$$

MONPREDOBJELIM $\langle \text{obj} \rangle P \vdash P$	MONPREDSUBJINTRO $P \vdash \langle \text{subj} \rangle P$	MONPREDOBJINTRO Objective P $P \vdash \langle \text{obj} \rangle P$	MONPREDSUBJELIM Objective P $\langle \text{subj} \rangle P \vdash P$
MONPREDOBJMONO $\frac{P \vdash Q}{\langle \text{obj} \rangle P \vdash \langle \text{obj} \rangle Q}$	MONPREDSUBJMONO $\frac{P \vdash Q}{\langle \text{subj} \rangle P \vdash \langle \text{subj} \rangle Q}$	MONPREDOBJIDEM $\langle \text{obj} \rangle \langle \text{obj} \rangle P \dashv\vdash \langle \text{obj} \rangle P$	
		MONPREDSUBJIDEM $\langle \text{subj} \rangle \langle \text{subj} \rangle P \dashv\vdash \langle \text{subj} \rangle P$	
MONPREDSUBJOBJADJ $\frac{\langle \text{subj} \rangle P \vdash Q}{P \vdash \langle \text{obj} \rangle Q}$	MONPREDOBJFORALL $\langle \text{obj} \rangle \forall x. \Phi x \dashv\vdash \forall x. \langle \text{obj} \rangle \Phi x$	MONPREDOBJEXIST $\exists x. \langle \text{obj} \rangle \Phi x \vdash \langle \text{obj} \rangle \exists x. \Phi x$	
MONPREDOBJAND $\langle \text{obj} \rangle P \wedge Q \dashv\vdash \langle \text{obj} \rangle P \wedge \langle \text{obj} \rangle Q$	MONPREDOBJOR $\langle \text{obj} \rangle P \vee \langle \text{obj} \rangle Q \vdash \langle \text{obj} \rangle P \vee Q$	MONPREDOBJSEPINTRO $\langle \text{obj} \rangle P * \langle \text{obj} \rangle Q \vdash \langle \text{obj} \rangle P * Q$	
MONPREDOBJSEP $\langle \text{obj} \rangle P * Q \dashv\vdash \langle \text{obj} \rangle P * \langle \text{obj} \rangle Q$	MONPREDOBJEMBED $\langle \text{obj} \rangle [P] \dashv\vdash [P]$	MONPREDOBJPURE $\langle \text{obj} \rangle \varphi \dashv\vdash \varphi$	
MONPREDOBJEMP $\langle \text{obj} \rangle \text{Emp} \dashv\vdash \text{Emp}$	MONPREDSUBJFORALL $\langle \text{subj} \rangle \forall x. \Phi x \vdash \forall x. \langle \text{subj} \rangle \Phi x$	MONPREDSUBJEXIST $\langle \text{subj} \rangle \exists x. \Phi x \dashv\vdash \exists x. \langle \text{subj} \rangle \Phi x$	
MONPREDSUBJAND $\langle \text{subj} \rangle P \wedge Q \vdash \langle \text{subj} \rangle P \wedge \langle \text{subj} \rangle Q$			
MONPREDSUBJOR $\langle \text{subj} \rangle P \vee Q \dashv\vdash \langle \text{subj} \rangle P \vee \langle \text{subj} \rangle Q$	MONPREDSUBJSEP $\langle \text{subj} \rangle P * Q \vdash \langle \text{subj} \rangle P * \langle \text{subj} \rangle Q$	MONPREDSUBJEMBED $\langle \text{subj} \rangle [P] \dashv\vdash [P]$	
MONPREDSUBJPERSISTENTLY $\langle \text{subj} \rangle \Box P \dashv\vdash \Box \langle \text{subj} \rangle P$		MONPREDOBJPERSISTENTLY $\langle \text{obj} \rangle \Box P \dashv\vdash \Box \langle \text{obj} \rangle P$	
MONPREDOBJPERSISTENT Persistent P $\frac{}{\text{Persistent}(\langle \text{obj} \rangle P)}$	MONPREDSUBJPERSISTENT Persistent P $\frac{}{\text{Persistent}(\langle \text{subj} \rangle P)}$	MONPREDOBJTIMELESS Timeless P $\frac{}{\text{Timeless}(\langle \text{obj} \rangle P)}$	
MONPREDSUBJTIMELESS Timeless P $\frac{}{\text{Timeless}(\langle \text{subj} \rangle P)}$			
MONPREDOBJPLAIN Plain P $\frac{}{\text{Plain}(\langle \text{obj} \rangle P)}$		MONPREDSUBJPLAIN Plain P $\frac{}{\text{Plain}(\langle \text{subj} \rangle P)}$	

Figure 4: Reasoning rules for objectively and subjectively.

$\frac{\text{MONPREDOBJECTIVEOBJ} \quad P \vdash \langle \text{obj} \rangle P}{\text{Objective } P}$	$\frac{\text{MONPREDOBJECTIVESUBJ} \quad \langle \text{subj} \rangle P \vdash P}{\text{Objective } P}$	$\frac{\text{MONPREDOBJECTIVEEMBED}}{\text{Objective } ([P])}$
$\frac{\text{MONPREDOBJECTIVEPURE} \quad \varphi \text{ pure}}{\text{Objective } (\varphi)}$	$\frac{\text{MONPREDOBJECTIVEEMP}}{\text{Objective } (\mathbf{Emp})}$	
$\frac{\text{MONPREDOBJECTIVEOBJMOD}}{\text{Objective } (\langle \text{obj} \rangle P)}$	$\frac{\text{MONPREDOBJECTIVESUBJMOD}}{\text{Objective } (\langle \text{subj} \rangle P)}$	
$\frac{\text{MONPREDOBJECTIVEAND} \quad \text{Objective } P \quad \text{Objective } Q}{\text{Objective } (P \wedge Q)}$	$\frac{\text{MONPREDOBJECTIVEOR} \quad \text{Objective } P \quad \text{Objective } Q}{\text{Objective } (P \vee Q)}$	
$\frac{\text{MONPREDOBJECTIVESEP} \quad \text{Objective } P \quad \text{Objective } Q}{\text{Objective } (P * Q)}$	$\frac{\text{MONPREDOBJECTIVEIMPL} \quad \text{Objective } P \quad \text{Objective } Q}{\text{Objective } (P \Rightarrow Q)}$	$\frac{\text{MONPREDOBJECTIVEWAND} \quad \text{Objective } P \quad \text{Objective } Q}{\text{Objective } (P \multimap Q)}$
$\frac{\text{MONPREDOBJECTIVEFORALL} \quad \forall x. \text{Objective } (\Phi x)}{\text{Objective } (\forall x. \Phi x)}$	$\frac{\text{MONPREDOBJECTIVEEXIST} \quad \forall x. \text{Objective } (\Phi x)}{\text{Objective } (\exists x. \Phi x)}$	
$\frac{\text{MONPREDOBJECTIVEPERSISTENTLY} \quad \text{Objective } P}{\text{Objective } (\Box P)}$		
$\frac{\text{MONPREDOBJECTIVELATER} \quad \text{Objective } P}{\text{Objective } (\triangleright P)}$	$\frac{\text{MONPREDOBJECTIVEEXCEPTZERO} \quad \text{Objective } P}{\text{Objective } (\Diamond P)}$	
$\frac{\text{MONPREDOBJECTIVEUPDATE} \quad \text{Objective } P}{\text{Objective } ({}_{E_1} \Vdash_{E_2} P)}$		

Figure 5: Rules for Objective propositions.

$\frac{\text{MONPREDATLEASTMONO} \quad w_1 \sqsubseteq w_2}{\uparrow w_2 \vdash \uparrow w_1}$	$\frac{\text{MONPREDATWORLDMONOP} \quad P \vdash Q}{@_w P \vdash @_w Q}$	$\frac{\text{MONPREDATWORLDMONOW} \quad w_1 \sqsubseteq w_2}{@_{w_1} P \vdash @_{w_2} P}$
$\text{MONPREDATSPILT} \quad P \vdash \exists w. @_w P * \uparrow w$	$\text{MONPREDATJOIN} \quad @_w P * \uparrow w \vdash P$	$\text{MONPREDATLEASTATWORLD} \quad @_w \uparrow w' \dashv\vdash [w' \sqsubseteq w]$
$\text{MONPREDATWORLDOBJECTIVE} \quad \text{Objective} (@_w P)$	$\frac{\text{MONPREDATWORLDPERSISTENT} \quad \text{Persistent } P}{\text{Persistent} (@_w P)}$	
$\frac{\text{MONPREDATWORLDTIMELESS} \quad \text{Timeless } P}{\text{Timeless} (@_w P)}$	$\frac{\text{MONPREDATWORLDPLAIN} \quad \text{Plain } P}{\text{Plain} (@_w P)}$	
$\text{MONPREDATLEASTPERSISTENT} \quad \text{Persistent} (\uparrow w)$	$\text{MONPREDATLEASTTIMELESS} \quad \text{Timeless} (\uparrow w)$	$\text{MONPREDATLEASTPLAIN} \quad \text{Plain} (\uparrow w)$

Figure 6: Rules for at-least and at-world.

$\text{oinv}_N(\Phi) \triangleq \lambda _ . [\text{inv}_N(\Phi \perp)]$		
$\frac{\text{MONPREDOINVALLOC} \quad \text{Objective } P}{\triangleright P \vdash \models \text{oinv}_N(P)}$	$\frac{\text{MONPREDOINVACC} \quad \text{Objective } P \quad \uparrow N \subseteq E}{\text{oinv}_N(P) \vdash_E \models_{E \uparrow N} (\triangleright P * (\triangleright P *_{E \uparrow N} \models_E \text{Emp}))}$	
$\text{MONPREDOINVERSISTENT} \quad \text{Persistent} (\text{oinv}_N(P))$	$\text{MONPREDOINVOBJECTIVE} \quad \text{Objective} (\text{oinv}_N(P))$	

Figure 7: Definition and rules for objective invariants.

Exercise 2.1. Try to prove (2) or (3). Where does the proof go wrong? Can you find a working alternative to (2) using $\uparrow w$ and $@_w P$?

Instead of trying harder to fix this issue (say, by requiring that the invariant be closed at the same world it was opened at), we'll for now commit to another workaround: We require a Φ in an $\text{oinv}_N(\Phi)$ to be objective. That way, we can avoid all these issues easily: Just evaluate the predicate in the invariant at the empty world! Ponder Figure 7 for the definition and rules for so-called *objective invariants*.

2.2 Why Monotone Predicates aren't enough

Let's try to start setting up a logical relation for region types in **MONPRED**. The type of worlds is the sets of regions $\mathcal{W} := \mathcal{P}(\text{RegName})$ with $\subseteq$ as a pre-order. Unfortunately, we'll quickly run into a rather major complication: $\mathcal{I}[-]$ is not actually a monotone predicate! If we have $\mathcal{I}[\sigma]$ and $\sigma \subseteq \sigma'$, then we certainly cannot deduce $\mathcal{I}[\sigma']$.

So plain monotone predicates give the right shape of propositions, but not yet the right way to tie the current world to the Iris resources owned by the logical relation.

$$\begin{aligned}
\text{cProp}_{\mathcal{W}}^{\mathcal{I}} &\triangleq \text{MONPRED}_{\mathcal{W}} \\
\mathcal{K}P &\triangleq \lambda w_1. \forall w_2 \sqsupseteq w_1. \mathcal{I}[w_2] \Rightarrow \exists w_3 \sqsupseteq w_2. \mathcal{I}[w_3] * P w_3 \\
\text{wp } e \{ \Phi \} &\triangleq \lambda w_1. \forall w_2 \sqsupseteq w_1. \mathcal{I}[w_2] \multimap \text{wp } e \{ v. \exists w_3 \sqsupseteq w_2. \mathcal{I}[w_3] * \Phi v w_3 \}.
\end{aligned}$$

$$\begin{array}{c}
\text{CROCUSIUPDMONO} \\
\frac{P \vdash Q}{\mathcal{K}P \vdash \mathcal{K}Q} \quad \text{CROCUSIUPDINTRO} \quad \text{CROCUSIUPDTRANS} \\
P \vdash \mathcal{K}P \quad \mathcal{K}(\mathcal{K}P) \vdash \mathcal{K}P \\
\text{CROCUSIUPDFRAME} \\
(\mathcal{K}P) * R \vdash \mathcal{K}(P * R) \\
\text{CROCUSWPIUPD} \quad \text{CROCUSIUPDWp} \\
\text{wp } e \{ v. \mathcal{K}\Phi v \} \vdash \text{wp } e \{ \Phi \} \quad \mathcal{K}\text{wp } e \{ \Phi \} \vdash \text{wp } e \{ \Phi \} \\
\text{CROCUSWPBIND} \\
\text{wp } e \{ v. \text{wp } K[v] \{ \Phi \} \} \vdash \text{wp } K[e] \{ \Phi \}
\end{array}$$

Figure 8: The CROCUS extension of MONPRED.

3 Crocus

This is where we deviate from the standard definition of monotone predicates in Iris (a la Mosel, iGPS, etc.), and define CROCUS. In this section, we introduce the basic ideas of KILR by extending the MONPRED logic from Section 2 with the *Kangaroo update modality*. The result is the CROCUS logic.

We denote the type of CROCUS propositions by $\text{cProp}_{\mathcal{W}}^{\mathcal{I}}$. Note that the logic itself is now parametrized by a world interpretation $\mathcal{I} : \mathcal{W} \rightarrow iProp$. CROCUS is defined like MONPRED_W, but with an additional connective: the *Kangaroo update modality* and a corresponding lifted weakest precondition (Figure 8).¹ With this definition, the current world w can be associated with the remaining ambient ghost state through $\mathcal{I}[-]$. This is what CROCUS is about!

4 A Kripke-Iris logical relation for region types

With CROCUS in hand, let's define a logical relation for the region type system. We choose $\mathcal{W} := \mathcal{P}(\text{RegName})$ with preorder $\subseteq$ and $\mathcal{I}[w] := \bigstar_{\rho \in \sigma} \text{is-alive } \rho$.

¹The trailing existentials are briefly explained at the end of this section, and silently ignored until then.

$$\begin{aligned}
\text{SemType} &\triangleq \{P : \text{Val} \rightarrow \text{cProp}_{\mathcal{W}}^T \mid \forall v. \text{Persistent } (P v) \wedge \text{Objective } (P v)\} \\
\llbracket \mathbb{1} \rrbracket v &\triangleq v = () \\
\llbracket A \times B \rrbracket v &\triangleq \exists v_1, v_2. v = (v_1, v_2) \wedge \llbracket A \rrbracket v_1 * \llbracket B \rrbracket v_2 \\
\llbracket A + B \rrbracket v &\triangleq (\exists u. v = \text{inj}_1 u \wedge \llbracket A \rrbracket u) \vee (\exists u. v = \text{inj}_2 u \wedge \llbracket B \rrbracket u) \\
\llbracket \text{ref}_\rho A \rrbracket v &\triangleq \exists \ell. v = (\rho, \ell) \wedge \text{oinv}_{(\rho, \ell)}(\exists u. \llbracket (\rho, \ell) \mapsto u \rrbracket * \llbracket A \rrbracket u) \\
\llbracket A \xrightarrow{\sigma} B \rrbracket v &\triangleq \forall v'. \uparrow \sigma * \llbracket A \rrbracket v' * \mathcal{E} \llbracket B \rrbracket (v v') \\
\mathcal{E} \llbracket A \rrbracket e &\triangleq \text{wp } e \{v. \llbracket A \rrbracket v\} \\
\mathcal{G} \llbracket \Gamma \rrbracket \gamma &\triangleq \bigstar_{x \in \text{dom } \Gamma} \llbracket \Gamma(x) \rrbracket (\gamma(x)) \\
\Gamma \models e : A @ \sigma &\triangleq \forall \gamma. \mathcal{G} \llbracket \Gamma \rrbracket \gamma * \uparrow \sigma * \mathcal{E} \llbracket A \rrbracket (\gamma(e))
\end{aligned}$$

Ponder the meaning of the invariant in the interpretation of $\text{ref}_\rho A$. Together with the generic Crocus WP rules in Figure 8, we can derive the following region-specific rules for $\text{wp } e \{\phi\}$:

$$\begin{array}{c}
\text{REGKILRWPREGION} \\
\frac{\forall \rho. \uparrow \{\rho\} * \text{wp } e \{v. \langle \text{obj} \rangle \Phi v\}}{\text{wp region } e \{\Phi\}}
\end{array}
\qquad
\begin{array}{c}
\text{REGKILRWALLOC} \\
\frac{\uparrow \{\rho\} * (\forall \ell. (\rho, \ell) \mapsto v * \Phi(\rho, \ell))}{\text{wp alloc}_\rho v \{\Phi\}}
\end{array}$$

$$\begin{array}{c}
\text{REGKILRWLOAD} \\
\frac{\uparrow \{\rho\} * (\rho, \ell) \mapsto v * ((\rho, \ell) \mapsto v * \Phi v)}{\text{wp } !(\rho, \ell) \{\Phi\}}
\end{array}
\qquad
\begin{array}{c}
\text{REGKILRWSTORE} \\
\frac{\uparrow \{\rho\} * (\rho, \ell) \mapsto v * ((\rho, \ell) \mapsto w * \Phi())}{\text{wp } (\rho, \ell) \leftarrow w \{\Phi\}}
\end{array}$$

Exercise 4.1. Take some time to appreciate these program logic rules. They're actually really nice! What we've built here as a side-product is a novel Iris program logic for region types.

We get:

Theorem 4.2 (Soundness). *If $\Gamma \vdash e : A @ \sigma$ then $\Gamma \models e : A @ \sigma$.*

Theorem 4.3 (Adequacy). *If $\emptyset \models e : A @ \emptyset$, then e is safe.*

Exercise 4.4. Ponder once more.

- (a) What properties does $\text{wp } e \{\phi\}$ have? What changed about the WP-rules? In what ways did they become simpler? Prove REGKILRWSTORE.
- (b) Prove the compatibility lemmas for REGTNEW, REGTMASK, and REGTSUB.
- (c) We require all semantic types to be objective. (Recall that objective propositions are those that do not depend on the world.) Why? Which typing rule requires that? And why does REGWPREGION require objectivity of Φ in the first place?
- (d) Prove adequacy (Theorem 4.3).
- (e) What did we get from defining a KILR instead of an LR for region types? What do you think of KILR at this point?

What’s your answer to Theorem 4.4.(e)? You might be thoroughly unimpressed. And that’s fair! It looks like we have taken a perfectly fine logical relation, and made it more complicated by adding an unnecessary layer of indirection.

But that’s not the point! The point of this section was to take a simple logical relation for an easy to understand type system, and look at what its KILR version would be. The interesting insight is that the logical relation looks *very* similar to a vanilla Iris logical relation for a standard type system with plain higher-order state, but with an interesting extra feature: regions! Indeed, almost all of the reasoning about regions (preserving $\mathcal{I}[w]$, etc.) is now hidden through the use of CROCUS: all a user has to deal with is $\uparrow w$, a very simple, well-behaved, persistent proposition. Soundness in the presence of this new, *world-dependent* proposition is ensured through a simple objectivity side condition on semantic types, stating that the validity of a value does not depend on the ambient world (even though *accessing* such a value might).

4.1 Looking ahead

We can follow a similar approach to define other logical relations, too. In Section 5, we’ll go through a similar pipeline: we start with an interesting type system, study it’s logical relation, and translate it to a KILR. There, you’ll get a much better impression for why the KILR approach is useful.

I conjecture that KILRs are also useful for many type systems that *restrict access to mutable state*, such as Pony (CITE), StackLang (from [4]), reachability types (CITE), and perhaps also Mezzo (CITE), capturing types (CITE), type systems with isolated regions (CITE), or other more expressive region constructs (CITE). I also suspect there are program logics (CITE wbpw, Cerise? crash-safety?) that can be understood in this setting.

I do not know whether Rust and its complicated (non well-bracketed!) borrowing discipline can be encoded here. I suspect that well-bracketed borrowing disciplines can.

Overall, I believe that the KILR-mindset has the following advantages:

- CROCUS is good at encoding certain patterns that separation logic is bad at encoding. Specifically, I’m thinking of different forms of restricted sharing: either in space (X may access me, but Y may not), or in time (X can access me, but only while it is running, not afterwards). This is common: It’s the key idea of most, if not all, region and data-race freedom type systems. CROCUS can reason about restricting access to mutable state, and can track how these access permissions evolve over time. I believe that this combination of worlds and separation logic for access control is under-explored.
- I believe that CROCUS allows for reasoning about compositions of such access-restricting logical features using *lenses* to focus parts on parts of the ambient state (cf. Section 12).
- I hope that KILR can guide logical relation development. For instance, I think that the ideas from the simple KILR for OxCaml with portability (Section 5) can inspire a logical relation for Pony, specifically of its reference capabilities.
- Perhaps KILR can even guide type system development? I have used logical relations to guide ideas about new type system primitives or unsafe functions. Maybe the KILR rules can help understand in-development type systems better, and make it easier to derive the right rules.
- I believe that CROCUS might simplify development of logical relations for well-bracketed program features.

4.2 Comparing the region types LR and KILR

After unfolding the CROCUS definitions, the LR and KILR for region types are mostly identical, except for one detail: the postcondition of the weakest precondition is allowed to grow the world, and hence the postcondition of the expression relation $\mathcal{E}[A]$ can.

$$\text{wp } e \{ \Phi \} \triangleq \lambda w. \forall w' \supseteq w. \mathcal{I}[w'] \multimap \text{wp } e \{ v. \exists w'' \supseteq w'. \mathcal{I}[w''] * \phi v \}$$

In essence, the proof still goes through because the postcondition of the weakest precondition in $\mathcal{E}[A]$ is objective. Regardless, something interesting is going on here: our previous logical relation accidentally encoded a property that is much stronger than what is needed to “just” prove soundness of the region type system. It encoded that before and after execution of an expression $\mathcal{E}[A] e$ the active regions σ must be *identical*, even though during execution the active region can temporarily grow using REGTMASK. Regions behave in a *well-bracketed* way.

In standard KLR, such behaviors are typically modelled by distinguishing between *public* and *private* transitions (TODO CITE). I believe that KILR can model and express well-bracketed reasoning principles, but it does require some additional infrastructure.

For the purpose of this region types system, we could have defined $\text{wp } e \{ \phi \}$ as

$$\text{wp } e \{ \Phi \} \triangleq \lambda w. \forall w' \supseteq w. \mathcal{I}[w'] \multimap \text{wp } e \{ v. \mathcal{I}[w'] * \phi v w' \}$$

and still obtain a reasonable KILR for region types. We’ll need the full expressivity of our actual definition of $\rightsquigarrow P$ and $\text{wp } e \{ \phi \}$ in Section 5, where it’s also useful for simplifying tedious proof steps.

5 A KILR logical relation for OCaml’s portability axis using thread-local invariants

Ben: Rewrite intro?

In this section, we’ll set up a KILR for PORTCAML, a simple version of OCaml with a single axis. Overall, our goal will be proving that the type system is sound and ensures data-race freedom.

To start, we introduce PORTCAML, and directly set up a logical relation for the language. The key difficulty will be ensuring that any reference can only be accessed from one thread at a time. Then, we start working on a KILR for PORTCAML. We stage its presentation: initially, we introduce a CROCUS instantiation with a bespoke notion of thread-local invariant. Specifically, we block thread-local invariants from moving between threads by adding an $\langle \text{obj} \rangle$ modality to the PORTWPFORK rule. The rules of this logic are then enough to state and prove soundness of the logical relation. Finally, we show how to define this logic, and extend it with two additional features: atomic references and locks.

After this section, you should read Section 6 on why I think that worlds are necessary to model OCaml and Section 7 on two contrasting two approaches to modelling OCaml’s modes.

5.1 Introduction to PORTCAML

This section introduces PORTCAML, an OCaml-style mode system with one modal axis. The axis controls where values may be used when programs spawn threads. It is incomparable

with OCaml’s portability and contention axes as described in CITE (cf. Section 15). We first describe the user-facing rules, then give the core type system and the shape of the program logic.

5.1.1 Portability and Contention

In PORTCAML, variables and the input and output types of functions are annotated with a *mode* from the lattice

$$\text{portable} < \text{nonportable} < \text{contended}.$$

The mode records the threads in which a value is valid. A **portable** value is valid in every thread. A **nonportable** value is valid only in the current thread, for instance because it carries a thread-local right to access mutable state. A **contended** value is valid in some thread, but the type system has forgotten which one.

Submoding follows the lattice. A **portable** value may be weakened to **nonportable** by specializing “every thread” to “the current thread”. A **nonportable** value may be weakened to **contended** by forgetting which thread it belongs to. Both weakenings discard information.

The default mode is **nonportable**. Sequential OCaml code runs in one thread, so unannotated values and arguments are treated as **nonportable**.

Modes are deep. For products (and records, analogously), pairing and projection preserve the same mode:

```
val pair : 'a @ 'm → 'b @ 'm → 'a * 'b @ 'm
val fst  : 'a * 'b @ 'm → 'a @ 'm
val snd  : 'a * 'b @ 'm → 'b @ 'm
```

Function capture. The central example is a parallel combinator:

```
val fork_join : (unit → 'a) @ portable → (unit → 'b) @ portable → 'a * 'b
```

Both thunks may run in fresh threads, so both must be portable closures. This requirement is mainly a restriction on what the closures capture.

The capture discipline has three cases:

1. A **portable** function preserves **portable** captures and sees all other captures as **contended**.
2. A **nonportable** function may capture anything.
3. A **contended** function may capture anything, but cannot be invoked.

A **portable** function may run in any thread. Therefore, every value it uses must be valid in the thread where it eventually runs. A captured **portable** value is valid everywhere. A captured **contended** value is inert: it cannot be read, written, or invoked, so it is harmless to carry around. A captured **nonportable** value is unsafe to use after migration, so the closure sees it only as **contended**.

For example, a closure that mutates a captured reference is necessarily **nonportable**:

```
let counter : int ref = ref 0
let bump : unit → unit = fun () → counter := !counter + 1
```

Here `counter` has the default mode, **nonportable**. The body of `bump` reads and writes it, so the closure must keep `counter` at **nonportable**. Passing `bump` to `fork_join` is rejected:

```
let () = fork_join bump (fun () → ())
```

Error: bump is nonportable even though it should be portable

Portability is not a ban on using thread-local objects. It is a ban on capturing thread-local objects and then carrying their access rights to another thread. For example, the load operation itself is portable:

```
val load : ('a ref → 'a) @@ portable
```

The function `load` can be called in any thread. The reference passed to it is checked at the call site, so a thread may call `load` on one of its own **nonportable** references. Similarly, **portable** functions can internally allocate and operate on mutable state and return it:

```
let f : (unit → int ref) @@ portable = let x = ref 0 in x := 1 ; x
```

Mutable references. Mutable references interact with modes through three rules:

1. References are never **portable**.
2. A **nonportable** reference behaves like an ordinary OCaml reference.
3. A **contended** reference cannot be read from or written to.

A mutable cell accessible from every thread would permit unsynchronized concurrent access. The basic operations therefore use the default **nonportable** mode:

```
val ref   : 'a → 'a ref
val (!)   : 'a ref → 'a
val (:=)  : 'a ref → 'a → unit
```

Sending a reference between threads is not forbidden. The act of sending weakens its mode to **contended**, after which neither side may access it through that value.

A closure that captures a mutable reference therefore has only two choices. It may stay **nonportable** and access the reference, or it may become **portable** and see the reference as **contended**. It cannot both travel between threads and keep the right to dereference.

Modalities. Deep modes are useful, but sometimes too coarse. Consider a record containing a portable callback and ordinary thread-local state:

```
type package = {
  run      : (unit → unit) @@ portable ;
  scratch  : int ref ;
}
```

To access `scratch`, the record as a whole must be **nonportable**, because `scratch` is a non-portable reference. If a copy of the record is passed to another thread, it becomes **contended**. Without the annotation on `run`, projecting the callback would also yield a **contended** function, which cannot be called. With `@@ portable`, the callback field is still projected at **portable**:

```
let access (p : package) = fork_join
  (fun _ → p.run ())
  (* this call ~~~~~ is fine because of the modality *)
```



```

(fun _ → p.scratch := 0)
~~~~~ (* but this access isn't *)
Error: p.scratch is contended even though it should be nonportable

```

5.1.2 Capsules as Isolated Regions

PORTCAML, in essence, ensures that memory is partitioned into disjoint *isolated regions*. There must not be pointers between isolated regions, only within them. The global invariant is that each isolated region is accessible from at most one thread at each point in time. We call these isolated regions *capsules*, in reference to the Capsule API (CITE DRFCaml, Section 2.something).

This gives a more concrete intuition for the modes. When code runs, it runs in some capsule:

1. A **portable** value is valid in all capsules: it does not refer to any capsule-dependent state.
2. A **nonportable** value is only valid in the currently running capsule.
3. A **contended** value is valid in some other capsule: it might refer to state owned by that capsule.

5.1.3 PORTCAML on paper

We now formalize the type system. The mode lattice is

$$M = \{\mathbf{portable}, \mathbf{nonportable}, \mathbf{contended}\}$$

with submoding $\mathbf{portable} \leq \mathbf{nonportable} \leq \mathbf{contended}$. We write $m \sqcup m'$ for the join and $m \sqcap m'$ for the meet.

Our typing judgement $\Gamma \vdash e : A @ m$ mentions modes in three places: the result mode, modal annotations inside types, and variable modes in Γ . The typing rules are collected in Figure 9.

Weakening. Submoding gives the weakening rule PORTTSUB. It allows an expression at mode m to be used at any weaker mode $m' \geq m$. There is no corresponding strengthening rule.

Functions The interesting rule is PORTTLAM, the introduction rule for λ -abstractions. Consider $\lambda x. e$ typed at mode m . Its function type is $A @ m_1 \rightarrow B @ m_2$ when the body has type B at mode m_2 under $x : A @ m_1$. The closure's own mode m also matters, because it determines how the closure may use free variables from Γ .

We express this constraint with a *lock* $\mathbf{lock}_m$ on contexts. The lock is applied before extending the context with the argument:

$$\begin{aligned}
\mathbf{lock}_m(\emptyset) &:= \emptyset \\
\mathbf{lock}_m(\Gamma, x : A @ m') &:= \begin{cases} \mathbf{lock}_m(\Gamma), x : A @ m' & \text{if } m \geq \mathbf{nonportable} \\ \mathbf{lock}_m(\Gamma), x : A @ m' & \text{if } m = m' = \mathbf{portable} \\ \mathbf{lock}_m(\Gamma), x : A @ \mathbf{contended} & \text{if } m = \mathbf{portable} \text{ and } m' \neq \mathbf{portable} \end{cases}
\end{aligned}$$

$$\begin{aligned}
A, B : \text{Type} &::= \mathbb{1} \mid A \times B \mid A + B \mid A @ m_1 \rightarrow B @ m_2 \mid \Box^\mu A \mid \text{ref } A \\
e : \text{Expr} &::= x \mid () \mid (e_1, e_2) \mid \pi_i e \mid \text{inj}_i e \\
&\mid \text{case } e \text{ of } \text{inj}_1 x \Rightarrow e_1 \mid \text{inj}_2 y \Rightarrow e_2 \\
&\mid \lambda x. e \mid e_1 e_2 \mid \text{box}_\mu e \mid \text{unbox } e \mid \text{alloc } e \mid !e \mid e_1 \leftarrow e_2 \mid \text{fork } e \\
m \in M &= \{\mathbf{portable}, \mathbf{nonportable}, \mathbf{contended}\} \\
\mu &\in \{\mu_{\mathbf{portable}}, \mu_{\mathbf{contended}}\} \\
\Gamma &::= \text{Var} \rightarrow_{\text{fin}} \text{Type} \times M
\end{aligned}$$

$\Gamma \vdash e : A @ m$

Ben: How does `notemathpar` interact with `and` and `newline` in `mathpartir`?

$ \begin{array}{c} \text{PORTTVAR} \\ \frac{(x : A @ m) \in \Gamma}{\Gamma \vdash x : A @ m} \end{array} $	$ \begin{array}{c} \text{PORTTUNIT} \\ \frac{}{\Gamma \vdash () : \mathbb{1} @ \mathbf{portable}} \end{array} $	$ \begin{array}{c} \text{PORTTPAIR} \\ \frac{\Gamma \vdash e_1 : A_1 @ m \quad \Gamma \vdash e_2 : A_2 @ m}{\Gamma \vdash (e_1, e_2) : A_1 \times A_2 @ m} \end{array} $
$ \begin{array}{c} \text{PORTTPROJ} \\ \frac{\Gamma \vdash e : A_1 \times A_2 @ m}{\Gamma \vdash \pi_i e : A_i @ m} \end{array} $		$ \begin{array}{c} \text{PORTTINJ} \\ \frac{\Gamma \vdash e : A_i @ m}{\Gamma \vdash \text{inj}_i e : A_1 + A_2 @ m} \end{array} $
$ \begin{array}{c} \text{PORTTCASE} \\ \frac{\Gamma \vdash e : A_1 + A_2 @ m \quad \Gamma, x : A_1 @ m \vdash e_1 : B @ m' \quad \Gamma, y : A_2 @ m \vdash e_2 : B @ m'}{\Gamma \vdash \text{case } e \text{ of } \text{inj}_1 x \Rightarrow e_1 \mid \text{inj}_2 y \Rightarrow e_2 : B @ m'} \end{array} $		
$ \begin{array}{c} \text{PORTTLAM} \\ \frac{\mathbf{lock}_m(\Gamma), x : A @ m_1 \vdash e : B @ m_2}{\Gamma \vdash \lambda x. e : (A @ m_1 \rightarrow B @ m_2) @ m} \end{array} $		
$ \begin{array}{c} \text{PORTTAPP} \\ \frac{\Gamma \vdash e_1 : (A @ m_1 \rightarrow B @ m_2) @ m_3 \quad \Gamma \vdash e_2 : A @ m_1 \quad m_3 \leq \mathbf{nonportable}}{\Gamma \vdash e_1 e_2 : B @ m_2} \end{array} $		
$ \begin{array}{c} \text{PORTTBOX} \\ \frac{\Gamma \vdash e : A @ \mu(m)}{\Gamma \vdash \text{box}_\mu e : \Box^\mu A @ m} \end{array} $	$ \begin{array}{c} \text{PORTTUNBOX} \\ \frac{\Gamma \vdash e : \Box^\mu A @ m}{\Gamma \vdash \text{unbox } e : A @ \mu(m)} \end{array} $	$ \begin{array}{c} \text{PORTTALLOC} \\ \frac{}{\Gamma \vdash \text{alloc } e : \text{ref } A @ \mathbf{nonportable}} \end{array} $
$ \begin{array}{c} \text{PORTTDEREF} \\ \frac{\Gamma \vdash e : \text{ref } A @ \mathbf{nonportable}}{\Gamma \vdash !e : A @ \mathbf{nonportable}} \end{array} $	$ \begin{array}{c} \text{PORTTASSIGN} \\ \frac{\Gamma \vdash e_1 : \text{ref } A @ \mathbf{nonportable} \quad \Gamma \vdash e_2 : A @ \mathbf{nonportable}}{\Gamma \vdash e_1 \leftarrow e_2 : \mathbb{1} @ \mathbf{nonportable}} \end{array} $	
$ \begin{array}{c} \text{PORTTFORK} \\ \frac{\Gamma \vdash e : \mathbb{1} @ \mathbf{portable}}{\Gamma \vdash \text{fork } e : \mathbb{1} @ \mathbf{portable}} \end{array} $		$ \begin{array}{c} \text{PORTTSUB} \\ \frac{\Gamma \vdash e : A @ m \quad m \leq m'}{\Gamma \vdash e : A @ m'} \end{array} $

Figure 9: PORTCAML typing judgement.

A **nonportable** closure leaves the context unchanged. A **portable** closure downgrades every **nonportable** value to **contended**. Thus, $\mu_{\text{portable}}(\Gamma)$ downgrades every binder that is not already **portable** to **contended**. Inside a **portable** closure, every captured **nonportable** reference is therefore **contended**.

A **contended** closure also leaves the context unchanged, but PORTTAPP prevents it from being called.

Modalities. A modality is a function $\mu : M \rightarrow M$ that can appear as a source-level field annotation. Two modalities are admissible:

$$\mu_{\text{portable}}(m) = \text{portable} \quad \mu_{\text{contended}}(m) = \text{contended}$$

Instead of formalizing records directly, the core type system models field modalities with the type constructor $\Box^\mu A$. The rules PORTTBOX and PORTTUNBOX are the introduction and projection principles for a modal field. Constructing $\Box^\mu A$ at mode m requires an A at mode $\mu(m)$. Eliminating $\Box^\mu A$ from an enclosing value at mode m returns an A at mode $\mu(m)$. This abstracts the source forms `A @@ portable` and `A @@ contended`.

Exercise 5.1. *Why is there no nonportable modality?*

5.1.4 Operational Semantics and Program Logic

The program logic mostly looks like the standard Iris program logic for a language with higher-order state. The main difference is that allocation, load, and store are not atomic. Internally, each operation takes several steps.

Those intermediate steps are where the operational semantics detects data races. When two threads access the same reference at the same time, their multi-step load or store protocols can overlap. The semantics detect this overlap and the offending thread crashes. The adequacy theorem for the PORTCAML weakest precondition therefore implies data-race freedom: well-typed programs never reach the race-detection crash state.

For more information, see DRFCaml CITE and RustBelt CITE.

5.2 A standard LR for PORTCAML

Let's first define a standard Iris logical relation for PORTCAML.

$$\begin{array}{c}
w \in \mathcal{T} \\
\text{tin}_-(-) : \text{namespace} \rightarrow \mathcal{T} \rightarrow (\mathcal{T} \rightarrow iProp) \rightarrow iProp \\
\mathcal{I}[-] : \mathcal{T} \rightarrow iProp \\
\\
\text{PORTLRWORLDALLOC} \quad \text{PORTLRTINVALLOC} \\
\frac{\forall w_1 \subseteq w_2. P w_1 \vdash P w_2}{\mathcal{I}[w_1] \multimap P w_1 \Rightarrow \exists w_2 \supseteq w_1. \mathcal{I}[w_2] \multimap \text{tin}_{w_2}^N(P)} \\
\\
\text{PORTLRTINVACC} \\
\frac{\uparrow N \subseteq E \quad \forall w_1 \subseteq w_2. P w_1 \vdash P w_2}{\mathcal{I}[w] \multimap \text{tin}_w^N(P) \Rightarrow_E P w \multimap (P w \Rightarrow_E \mathcal{I}[w])}
\end{array}$$

Figure 10: Iris rules for token invariants.

$$\begin{aligned}
\text{SemType} &\triangleq \{P : \text{Val} \rightarrow M \rightarrow \mathcal{T} \rightarrow iProp \mid \\
&\quad \bigwedge \left(\begin{array}{l} \forall v m w. \text{Persistent}(P_m^w v), \\ \forall v m_1 m_2 w_1 w_2. m_1 \leq m_2 \implies w_1 \subseteq w_2 \implies \\ \quad P_{m_1}^{w_1} v \vdash P_{m_2}^{w_2} v, \\ \forall v m w_1 w_2. m = \mathbf{portable} \vee m = \mathbf{contended} \implies \\ \quad P_m^{w_1} v \vdash P_m^{w_2} v \end{array} \right) \} \\
\llbracket \mathbf{1} \rrbracket_m^w v &\triangleq v = () \\
\llbracket A \times B \rrbracket_m^w v &\triangleq \exists v_1 v_2. v = (v_1, v_2) \wedge \llbracket A \rrbracket_m^w v_1 \multimap \llbracket B \rrbracket_m^w v_2 \\
\llbracket A + B \rrbracket_m^w v &\triangleq (\exists u. v = \text{inj}_1 u \wedge \llbracket A \rrbracket_m^w u) \vee (\exists u. v = \text{inj}_2 u \wedge \llbracket B \rrbracket_m^w u) \\
\llbracket A @ m' \rrbracket_m^w v &\triangleq \llbracket A \rrbracket_{m'}^w v \\
\llbracket \text{ref } A \rrbracket_m^w v &\triangleq \begin{cases} \exists \ell. v = \ell & \text{if } m = \mathbf{contended} \\ \exists \ell. v = \ell \wedge \\ \quad \text{tin}_w^\ell(\lambda w'. \exists u. \ell \mapsto u \multimap \llbracket A \rrbracket_{\mathbf{nonportable}}^{w'} u) & \text{if } m = \mathbf{nonportable} \\ \perp & \text{if } m = \mathbf{portable} \end{cases} \\
\llbracket A @ m_1 \rightarrow B @ m_2 \rrbracket_m^w v &\triangleq \begin{cases} \text{Emp} & \text{if } m = \mathbf{contended} \\ \Box \forall w_2 \supseteq w. \forall u. \llbracket A \rrbracket_{m_1}^{w_2} u \multimap \mathcal{E}[\llbracket B \rrbracket_{m_2}^{w_2}(v u)] & \text{if } m = \mathbf{nonportable} \\ \Box \forall w_2 u. \llbracket A \rrbracket_{m_1}^{w_2} u \multimap \mathcal{E}[\llbracket B \rrbracket_{m_2}^{w_2}(v u)] & \text{if } m = \mathbf{portable} \end{cases} \\
\mathcal{E}[\llbracket A \rrbracket_m^w e] &\triangleq \forall w_2 \supseteq w. \mathcal{I}[w_2] \multimap \text{wp } e \{v. \mathcal{I}[w_2] \multimap \llbracket A \rrbracket_m^{w_2} v\} \\
\mathcal{G}[\Gamma]^w \gamma &\triangleq \bigstar_{(x:A@m) \in \Gamma} \llbracket A \rrbracket_m^w(\gamma(x)) \\
\Gamma \models e : A @ m &\triangleq \forall w \gamma. \mathcal{G}[\Gamma]^w \gamma \multimap \mathcal{E}[\llbracket A \rrbracket_m^w(\gamma(e))]
\end{aligned}$$

What the hell happened here? This looks terrible! Let’s walk through this very slowly, and figure out what all these parts mean and how they fit together. We’ll go through the definitions step-by-step.

Worlds Recall the capsule intuition from Section 5.1.2. How do we model the notion of capsule, and intuition like “code runs in some capsule” and “values are only valid in this capsule”? Recall the Iris logical relation for region types from Section 1, and specifically its expression relation: It is parametrized by a set of regions σ , and this parameter is associated with both the type system (that mentions σ in functions and typing judgements).

We’re in a very different situation: **PORTCAML** doesn’t represent/track the current capsule; it only tracks the relationships between capsules by implicit quantification, in some sense.

The logical relation for **PORTCAML** makes this implicit quantification explicit by parametrizing the value and expression relations over a world $w \in \mathcal{T}$. Intuitively, $\mathcal{T}$ can be thought of as a (representation of) the set of references the value/the expression can access (transitively).

Ben: there’s some insight in why region types got to have objective semtypes. why?

Ben: Figure out how much more I have to say in Section 6.

Semantic Types Semantic types are predicates on values, modes, and *worlds*.

Explain standard LR.

Axiomatize token invariants, and justify why we need them.

(Give definition of token invariants, but don’t explain it much)

Give example proof: Lots of $\subseteq$!

To conclude this section: this logical relation is annoying to define, difficult to understand, and nasty to reason about. Can we do better?

5.3 Crocus logic

Ben: rewrite this: Based on *tin* from previous section, we define invariants in Crocus. Write down the actual rules with the right, masked properties, and defer it’s definition to a later section

First and foremost, we must prevent the same thread-local invariant from being opened twice. We’ll use the same mechanism Iris uses: masks. Generalising $\rightsquigarrow P$ to masks is actually relatively simple. We start by parametrizing the world interpretation by a mask $\mathcal{I}[w]^E$. Now, we can define a *fancy Kangaroo update*:

$$\rightsquigarrow_{E_1} P \triangleq \lambda w. \forall w' \sqsubseteq w. \mathcal{I}[w']^{E_1} \multimap_{\top} \exists w'' \sqsubseteq w'. \mathcal{I}[w'']^{E_2} * P w''$$

Note that we chose the *inner* fancy update to have masks $\top, \top$. I have not worked the consequences of this choice in detail; It has worked so far.

Now, ponder Figure 11. It presents an instantiation of *cProp* with thread-local invariants *tin*. So, specifically:

- Thread-local invariants can be opened non-atomically using **PORTWPIUPD**. Note that this doesn’t mean that we can open standard (objective) invariants non-atomically: an indexed update contains a fancy update at *full* mask.

$$\begin{array}{c}
\text{PORTTINVALLOC} \\
\frac{P \rightsquigarrow_E \exists \gamma. \gamma \in \uparrow N \wedge \text{tinv } \gamma \ N \ P}{\text{wp fork } e \{v. v = ()\}}
\end{array}
\quad
\begin{array}{c}
\text{PORTTINVACC} \\
\frac{\gamma \in E}{\text{tinv } \gamma \ N \ P \rightsquigarrow_{E \setminus \{\gamma\}} P * (\forall E'. P \rightsquigarrow_{E' \cup \{\gamma\}} \text{Emp})}
\end{array}$$

$$\begin{array}{c}
\text{PORTWPFORK} \\
\frac{\langle \text{obj} \rangle \text{wp } e \{ \phi \}}{\text{wp fork } e \{v. v = ()\}}
\end{array}
\quad
\begin{array}{c}
\text{PORTWPIUPD} \\
\frac{E \subseteq E' \quad E' \rightsquigarrow_E \text{wp } e \{v. \rightsquigarrow_{E'} \phi e\}}{\text{wp}_{E'} e \{ \phi \}}
\end{array}$$

Figure 11: PortCaml thread-local invariant rules.

- Thread-local invariants can store arbitrary propositions, and behave very similar to invariants in standard Iris. Note, however, that they are not objective.
- Thread-local invariants are parametrized by a name γ . This is to simplify the model we'll give in Section 5.5. A “better” (in some ways, but not in others) model of thread-local invariants is presented in Section 10.

We'll use `tinv` for modeling standard references, and standard objective invariants for atomic references.

5.4 KILR

We define the *mode-modality*:

$$\bigcirc^m P := \begin{cases} \langle \text{subj} \rangle P & \text{if } m = \mathbf{contended} \\ P & \text{if } m = \mathbf{nonportable} \\ \langle \text{obj} \rangle P & \text{if } m = \mathbf{portable} \end{cases}$$

Ponder the logical relation.

$$\begin{aligned}
\text{SemType} &\triangleq \{P : \text{cProp} \mid \text{Persistent } P\} \\
\llbracket \mathbf{1} \rrbracket v &\triangleq v = () \\
\llbracket A \times B \rrbracket v &\triangleq \exists v_1, v_2. v = (v_1, v_2) \wedge \llbracket A \rrbracket v_1 * \llbracket B \rrbracket v_2 \\
\llbracket A + B \rrbracket v &\triangleq (\exists u. v = \text{inj}_1 u \wedge \llbracket A \rrbracket u) \vee (\exists u. v = \text{inj}_2 u \wedge \llbracket B \rrbracket u) \\
\llbracket \text{ref } A \rrbracket v &\triangleq \exists \ell \gamma. v = \ell \wedge \text{tinv } \gamma \top (\exists u. (\rho, \ell) \mapsto u * \llbracket A \rrbracket u) \\
\llbracket A @ m \rrbracket v &\triangleq \bigcirc^m (\llbracket A \rrbracket v) \\
\llbracket A @ m_1 \rightarrow B @ m_2 \rrbracket v &\triangleq \Box \forall u. \bigcirc^{m_1} (\llbracket A \rrbracket u) \multimap \mathcal{E} \llbracket B \rrbracket^{m_2} (v u) \\
\mathcal{E} \llbracket A \rrbracket^m e &\triangleq \text{wp } e \{v. \bigcirc^m (\llbracket A \rrbracket v)\} \\
\mathcal{G} \llbracket \Gamma \rrbracket \gamma &\triangleq \bigstar_{(x:A@m) \in \Gamma} \bigcirc^m (\llbracket A \rrbracket (\gamma(x))) \\
\Gamma \models e : A @ m &\triangleq \forall \gamma. \mathcal{G} \llbracket \Gamma \rrbracket \gamma \multimap \mathcal{E} \llbracket A \rrbracket^m (\gamma(e))
\end{aligned}$$

Lemma 5.2. *The proposition $\mathcal{G} \llbracket \mathbf{1}_m \Gamma \rrbracket \gamma$ is objective if $m = \mathbf{portable}$.*

Theorem 5.3. *FTLR*

Theorem 5.4. *ADEQUACY*

The proofs are all very standard. Some of them require a case distinction on a mode m , and then follow by standard logical relations arguments and the introduction/elimination rules for objectively and subjectively.

5.5 Model of the Crocus logic

We choose $\mathcal{W} := \mathcal{P}(GName)$ ordered by $\subseteq$. We define:

Ben: There's a lie here: γ refers to two gnames! How to best denote this here?

$$\begin{aligned} \text{tin} \gamma N P &\triangleq \gamma \in \uparrow N \wedge \uparrow \{\gamma\} \wedge \text{inv } N (\exists w. [\gamma \hookrightarrow^{1/2} w] * (@_w P * [\text{excl}()]^\gamma \vee \gamma \hookrightarrow^{1/2} w)) \\ \mathcal{I}[w]^E &\triangleq \bigstar_{\gamma \in w} \gamma \notin E \vee \gamma \in E \wedge \exists w' \sqsubseteq w. \gamma \hookrightarrow^{1/2} w' \end{aligned}$$

5.6 Adding atomic references

5.7 Adding locks

6 Why worlds are inevitable

7 How to model modes?

8 MonPred vs ParamPred

Ben: Do I want to write this section?

9 OPEN: A logical relation for PortSharedCaml

I do not know yet how to model read-only shared state in Crocus.

10 Thread-local invariants and masks

How to get the right rules wrt masks and the masked Kangaroo update (Figure 12).

11 TODO: A KILR logical relation for OCaml's borrowing using local invariants

I haven't actually worked this out! The key question is how to model the local mode vs the affinity modes, and which approach to use for each.

$$\begin{array}{c}
\text{MASKEDTINVALLOC} \\
\frac{P \xrightarrow{\text{fork}}_{\top} \text{tin}v N P}{\text{wp fork } e \{v. v = ()\}} \\
\\
\text{MASKEDTINVACC} \\
\frac{\gamma \in E}{\text{tin}v N P \xrightarrow{\text{fork}}_{E \setminus \uparrow N} P * (\forall E'. P \xrightarrow{\text{fork}}_{E' \cup \uparrow N} \text{Emp})} \\
\\
\text{MASKEDPORTWPFORK} \\
\frac{\langle \text{obj} \rangle \text{wp } e \{ \phi \}}{\text{wp fork } e \{v. v = ()\}} \\
\\
\text{MASKEDPORTWPIUPD} \\
\frac{E \subseteq E' \quad E' \xrightarrow{\text{fork}}_E \text{wp } e \{v. E' \xrightarrow{\text{fork}}_{E'} \phi e\}}{\text{wp}_{E'} e \{ \phi \}}
\end{array}$$

Figure 12: Masked thread-local invariant rules.

12 KILR and lenses

I have worked out the technical details of a simple CROCUS instantiation with a product of worlds and masked interpretations. I haven't tried verifying a concrete type system.

13 TODO: KILR and public/private transitions

This will be cool!

14 TODO: A KILR logical relation for OCaml local state

Example for public-private state. Will be necessary for OCaml!

15 TODO: OCaml with split axes

Is this part of the same project? Not sure yet.

16 AME, and what does it have to do with anything

Ben: Write about AME, and how it relates to the other stuff

Ben: KILR for substructural modes? Could be a cool project

Figure 13 describes an interesting dual-context affine type system for a language with higher-order state.² The type system tracks duplicable vs. one-shot computations ($A(\tau \rightarrow \sigma)$ vs. $\tau \rightarrow \sigma$) as well as unique vs. aliased references ($\text{ref } \tau$ vs. $E(\text{ref } \tau)$) in the same system. Not many other systems support both these features at once. Unique references can be strongly updated (AMETREAD , AMETSET), and aliased references behave like regular references (AMETREADE , AMETSETT).

AMELANG supports the following modalities:

- A *state* monad M . All stateful operations lie in here. Execution in M is delayed: the execution of ref , load , store , and bind is delayed, and the language itself does not have a forcing construct.

²An equivalent type system can be obtained from an OCaml-style substructural type system with a single mode axis **many** < **once** < **aliased**.

- A *duplicable* comonad A , analogously to linear-logic's $!$.
- A *duplicated* monad E . All objects in here can be duplicated, regardless of whether they are linear or not. Execution in E is immediate, i.e. not delayed.

All operations on E and A (box, letbox $x = e_1$ in e_2 , share, letshare $x = e_1$ in e_2 , dup, peel) are no-ops. We have the following equations:

- $E(A P) = A P$. That is, duplicating a duplicable object is a no-op. It is operationalized by the no-op peel.
- $E P = A(E P)$. That is, a duplicated object is duplicable. It is operationalized by the no-op dup.

The type system is sound:

Theorem 16.1. *If $\emptyset; \emptyset \vdash e : A$ then e is safe.*

Theorem 16.2. *If $\emptyset; \emptyset \vdash e : M A$ then force e is safe.*

What's interesting about this type system is that has modalities that track uniqueness and linearity in one system, and that it has very strong rules (dup and peel) for how to reason about these modalities.

There is at least one other system that also tries to combine uniqueness and linearity: LCU [3]. It follows an entirely different approach: LCU starts with a linear system, and adds a unique modality $*$ on top. Duplicable objects $!A$ can be copied to obtain unique objects $*A$, and unique objects can later aliased to $!A$. From a mode-al point of view, LCU has modes **unique** < **many/aliased** < **once**.

AME, on the other hand, has modes **many** < **once/unique** < **aliased**. This allows AME to have deeply unique references $\text{ref}(\text{ref } \tau)$ (would correspond to $*\text{ref}(*\text{ref } \tau)$, which I think cannot be allocated in LCU) and references to one-shot computations $\text{ref}(\tau \rightarrow M \sigma)$, neither of which, I believe, can be expressed in LCU.

LCU was recently extended with borrowing [2]. This calculus does support deeply unique references, but forbids any form of computation in unique references. (Recall that LCU can store multi-shot computations in unique references.)

There's also this very interesting paper [1], which sounds like it might be related, but not too closely?

The state monad plays a very important role in ensuring that A and E are well-behaved (i.e., a comonad/monad, respectively). Indeed, without a state monad, A couldn't support axiom K:

$$\text{alloc} : A(\mathbb{1} \rightarrow \text{ref } \mathbb{1}) \approx A \mathbb{1} \rightarrow A(\text{ref } \mathbb{1})$$

This is unsound, because we must never have $A(\text{ref } \tau)$: we could duplicate it, then weaken the A away, and strongly update one of the copies of the reference. However, with a state monad:

$$\text{alloc} : A(\mathbb{1} \rightarrow M(\text{ref } \mathbb{1})) \approx A \mathbb{1} \rightarrow A(M(\text{ref } \mathbb{1}))$$

This just says that we can obtain the duplicable permission to allocate new references because computations in M are delayed.

This has the interesting consequence that we must not allow a monad under an E to be executed: the semantic interpretation of $E(M \tau)$ is vacuously true. Indeed, this is important: we can construct a loop-free program that returns objects of type $M(E(M(\text{ref } \mathbb{1} \times \text{ref } \mathbb{1})))$ where both references will be the same!

Exercise 16.3. *Write that program.*

However, because we can never execute this computation, this does not lead to an unsoundness issue. Technically, we might even be able to write a terminating program of type $E(M(\perp))$, but that doesn't mean the type system is unsound.

References

- [1] Hemant Gouni, Frank Pfenning, and Jonathan Aldrich. “Security Reasoning via Substructural Dependency Tracking”. In: *Proc. ACM Program. Lang.* 10.POPL (2026), pp. 777–805. DOI: 10.1145/3776669. URL: <https://doi.org/10.1145/3776669>.
- [2] Danielle Marshall and Dominic Orchard. “Functional Ownership through Fractional Uniqueness”. In: *Proc. ACM Program. Lang.* 8.OOPSLA1 (2024), pp. 1040–1070. DOI: 10.1145/3649848. URL: <https://doi.org/10.1145/3649848>.
- [3] Danielle Marshall, Michael Vollmer, and Dominic Orchard. “Linearity and Uniqueness: An Entente Cordiale”. In: *Programming Languages and Systems - 31st European Symposium on Programming, ESOP 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings*. Ed. by Ilya Sergey. Lecture Notes in Computer Science. Springer, 2022, pp. 346–375. DOI: 10.1007/978-3-030-99336-8_13. URL: https://doi.org/10.1007/978-3-030-99336-8_13.
- [4] Simon Friis Vindum, Aina Linn Georges, and Lars Birkedal. “The Nextgen Modality: A Modality for Non-Frame-Preserving Updates in Separation Logic”. In: *Proceedings of the 14th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2025, Denver, CO, USA, January 20-21, 2025*. Ed. by Kathrin Stark et al. ACM, 2025, pp. 83–97. DOI: 10.1145/3703595.3705876. URL: <https://doi.org/10.1145/3703595.3705876>.

$\tau, \sigma, \rho ::= \mathbb{1} \mid \tau \rightarrow \sigma \mid \tau \times \sigma \mid \tau + \sigma \mid \text{ref } \tau \mid \mathbf{A} \tau \mid \mathbf{E} \tau \mid \mathbf{M} \tau$
 $e ::= v \mid \mathbf{x} \mid x \mid \lambda \mathbf{x}. e \mid e_1 e_2 \mid (e_1, e_2) \mid \text{let } (\mathbf{x}, \mathbf{y}) = e_1 \text{ in } e_2 \mid \text{inl } e \mid \text{inr } e \mid \text{case } e \text{ of } \text{inl } \mathbf{x} \Rightarrow e_1 \mid \text{inr } \mathbf{y} \Rightarrow e_2$
 $\mid \text{letbox } x = e_1 \text{ in } e_2 \mid \text{alloc } e \mid ! e \mid e_1 \leftarrow e_2$
 $\mid \text{box } e \mid \text{share } e \mid \text{letshare } \mathbf{x} = e_1 \text{ in } e_2 \mid \text{dup } e \mid \text{peel } e \mid \text{distA } e \mid \text{distE } e$
 $\mid \text{return } e \mid \text{bind } \mathbf{x} = e_1 \text{ in } e_2 \mid \text{alloc } e \mid ! e \mid \text{readE } e \mid e_1 \leftarrow e_2 \mid \text{setT } e_1 \leftarrow e_2$
 $v ::= () \mid \ell \mid \lambda \mathbf{x}. e \mid (v_1, v_2) \mid \text{inl } v \mid \text{inr } v$
 $x, y, z \in \text{Var}_s, \quad \Delta \in \text{Var}_s \rightarrow_{\text{fin}} \tau.$
 $\mathbf{x}, \mathbf{y}, \mathbf{z} \in \text{Var}_\ell, \quad \Gamma \in \text{Var}_\ell \rightarrow_{\text{fin}} \tau.$
 $\Gamma_1 \uplus \Gamma_2$ is defined when $\text{dom}(\Gamma_1) \cap \text{dom}(\Gamma_2) = \emptyset$,
 and then denotes $\Gamma_1 \cup \Gamma_2$.

$\frac{\text{AMETUNIT}}{\Delta; \Gamma \vdash () : \mathbf{1}}$		$\frac{\text{AMETVARGAMMA} \quad \Gamma(\mathbf{x}) = \tau}{\Delta; \Gamma \vdash \mathbf{x} : \tau}$		$\frac{\text{AMETVARDELTA} \quad \Delta(x) = \tau}{\Delta; \Gamma \vdash x : \tau}$	
$\frac{\text{AMETFUN} \quad \Delta; \Gamma, \mathbf{x} : \tau \vdash e : \sigma}{\Delta; \Gamma \vdash \lambda \mathbf{x}. e : \tau \rightarrow \sigma}$		$\frac{\text{AMETAPP} \quad \Delta; \Gamma_1 \vdash e_1 : \tau \rightarrow \sigma \quad \Delta; \Gamma_2 \vdash e_2 : \tau}{\Delta; \Gamma_1 \uplus \Gamma_2 \vdash e_1 e_2 : \sigma}$			
$\frac{\text{AMETPAIR} \quad \Delta; \Gamma_1 \vdash e_1 : \tau \quad \Delta; \Gamma_2 \vdash e_2 : \sigma}{\Delta; \Gamma_1 \uplus \Gamma_2 \vdash (e_1, e_2) : \tau \times \sigma}$		$\frac{\text{AMETTLETPAIR} \quad \Delta; \Gamma_1 \vdash e_1 : \tau \times \sigma \quad \Delta; \Gamma_2, \mathbf{x} : \tau, \mathbf{y} : \sigma \vdash e_2 : \rho}{\Delta; \Gamma_1 \uplus \Gamma_2 \vdash \text{let } (\mathbf{x}, \mathbf{y}) = e_1 \text{ in } e_2 : \rho}$		$\frac{\text{AMETINJL} \quad \Delta; \Gamma \vdash e : \tau}{\Delta; \Gamma \vdash \text{inl } e : \tau + \sigma}$	
$\frac{\text{AMETINJR} \quad \Delta; \Gamma \vdash e : \sigma}{\Delta; \Gamma \vdash \text{inr } e : \tau + \sigma}$		$\frac{\text{AMETCASE} \quad \Delta; \Gamma_1 \vdash e : \tau + \sigma \quad \Delta; \Gamma_2, \mathbf{x} : \tau \vdash e_1 : \rho \quad \Delta; \Gamma_2, \mathbf{y} : \sigma \vdash e_2 : \rho}{\Delta; \Gamma_1 \uplus \Gamma_2 \vdash \text{case } e \text{ of } \text{inl } \mathbf{x} \Rightarrow e_1 \mid \text{inr } \mathbf{y} \Rightarrow e_2 : \rho}$			
$\frac{\text{AMETBOX} \quad \Delta; \emptyset \vdash e : \tau}{\Delta; \Gamma \vdash \text{box } e : \mathbf{A} \tau}$		$\frac{\text{AMETTLETBOX} \quad \Delta; \Gamma_1 \vdash e_1 : \mathbf{A} \tau \quad \Delta, x : \tau; \Gamma_2 \vdash e_2 : \sigma}{\Delta; \Gamma_1 \uplus \Gamma_2 \vdash \text{letbox } x = e_1 \text{ in } e_2 : \sigma}$			
$\frac{\text{AMETSHARE} \quad \Delta; \Gamma \vdash e : \tau}{\Delta; \Gamma \vdash \text{share } e : \mathbf{E} \tau}$		$\frac{\text{AMETLETSHARE} \quad \Delta; \Gamma_1 \vdash e_1 : \mathbf{E} \tau \quad \Delta; \Gamma_2, \mathbf{x} : \tau \vdash e_2 : \mathbf{E} \sigma}{\Delta; \Gamma_1 \uplus \Gamma_2 \vdash \text{letshare } \mathbf{x} = e_1 \text{ in } e_2 : \mathbf{E} \sigma}$		$\frac{\text{AMETDUP} \quad \Delta; \Gamma \vdash e : \mathbf{E} \tau}{\Delta; \Gamma \vdash \text{dup } e : \mathbf{A}(\mathbf{E} \tau)}$	
				$\frac{\text{AMETPEEL} \quad \Delta; \Gamma \vdash e : \mathbf{E}(\mathbf{A} \tau)}{\Delta; \Gamma \vdash \text{peel } e : \mathbf{A} \tau}$	
$\frac{\text{AMETDISTA} \quad \Delta; \Gamma \vdash e : \mathbf{A}(\tau + \sigma)}{\Delta; \Gamma \vdash \text{distA } e : (\mathbf{A} \tau) + (\mathbf{A} \sigma)}$		$\frac{\text{AMETDISTE} \quad \Delta; \Gamma \vdash e : \mathbf{E}(\tau + \sigma)}{\Delta; \Gamma \vdash \text{distE } e : (\mathbf{E} \tau) + (\mathbf{E} \sigma)}$			
$\frac{\text{AMETRETURN} \quad \Delta; \Gamma \vdash e : \tau}{\Delta; \Gamma \vdash \text{return } e : \mathbf{M} \tau}$		$\frac{\text{AMETBIND} \quad \Delta; \Gamma_1 \vdash e_1 : \mathbf{M} \tau \quad \Delta; \Gamma_2, \mathbf{x} : \tau \vdash e_2 : \mathbf{M} \sigma}{\Delta; \Gamma_1 \uplus \Gamma_2 \vdash \text{bind } \mathbf{x} = e_1 \text{ in } e_2 : \mathbf{M} \sigma}$			
$\frac{\text{AMETNEW} \quad \Delta; \Gamma \vdash e : \tau}{\Delta; \Gamma \vdash \text{alloc } e : \mathbf{M}(\text{ref } \tau)}$		$\frac{\text{AMETNEW E} \quad \Delta; \Gamma \vdash e : \mathbf{E} \tau}{\Delta; \Gamma \vdash \text{alloc } e : \mathbf{M}(\mathbf{E}(\text{ref } \tau))}$		$\frac{\text{AMETREAD} \quad \Delta; \Gamma \vdash e : \text{ref } \tau}{\Delta; \Gamma \vdash ! e : \mathbf{M} \tau}$	
				$\frac{\text{AMETREADE} \quad \Delta; \Gamma \vdash e : \mathbf{E}(\text{ref } \tau)}{\Delta; \Gamma \vdash \text{readE } e : \mathbf{M}(\mathbf{E} \tau)}$	
$\frac{\text{AMETSET} \quad \Delta; \Gamma_1 \vdash e_1 : \text{ref } \tau \quad \Delta; \Gamma_2 \vdash e_2 : \sigma}{\Delta; \Gamma_1 \uplus \Gamma_2 \vdash e_1 \leftarrow e_2 : \mathbf{M}(\text{ref } \sigma)}$		$\frac{\text{AMETSET T} \quad \Delta; \Gamma_1 \vdash e_1 : \mathbf{E}(\text{ref } \tau) \quad \Delta; \Gamma_2 \vdash e_2 : \mathbf{E} \tau}{\Delta; \Gamma_1 \uplus \Gamma_2 \vdash \text{setT } e_1 \leftarrow e_2 : \mathbf{M}(\mathbf{E}(\text{ref } \tau))}$			

Figure 13: Surface syntax and typing judgement for AME.